SOC Automation Project

Automated Threat Detection, Enrichment and Response using Wazuh, Shuffle and TheHive

Prepared by: Bibash Bhandari, St. Xavier's College, Maitighar

Project type: Cybersecurity lab / Security Operations Center (SOC) automation

Keywords: SIEM, SOAR, Wazuh, Shuffle, TheHive, Sysmon, MITRE ATT&CK, VirusTotal, incident response

1. Project Overview

1.1 Objective

The objective of this project was to design and build a small SOC automation pipeline that can detect a malicious tool on a Windows endpoint, enrich the alert with threat intelligence, open a case for the analyst, notify the analyst by email, and carry out a response action once the analyst approves it. The aim was to reduce the manual, repetitive work an analyst normally does when triaging an alert.

1.2 Summary

A Windows 10 client running Sysmon and the Wazuh agent sends its logs to a Wazuh Manager. A custom detection rule identifies the execution of Mimikatz, a well-known credential-dumping tool (MITRE ATT&CK technique T1003). When the rule fires, a Wazuh integration forwards the alert to Shuffle, an open-source SOAR platform, through a webhook. Shuffle extracts the file hash, checks it against VirusTotal, creates an alert in TheHive (case management), and emails the SOC analyst asking whether the source IP should be blocked. Based on the analyst's answer, Shuffle instructs Wazuh to run an active-response command that blocks the IP on the endpoint's firewall.

1.3 Goals

- Collect and centralise endpoint logs from a Windows 10 machine in a SIEM.
- Write a custom detection rule mapped to the MITRE ATT&CK framework.
- Automate alert triage: IOC enrichment, case creation and analyst notification.
- Implement a human-in-the-loop response so that blocking actions are approved by an analyst.

1.4 Project basis

The architecture and workflow of this project follow the publicly available MyDFIR SOC Automation project walkthrough, which is cited in the References section. This document records the components that were configured, how they were connected, and what was observed at each stage.

2. Tools and Technologies

Component	Role in the project
Windows 10 client	Monitored endpoint; the machine on which the test attack was simulated.
Sysmon	Provides detailed process-creation telemetry (Event ID 1) including file hashes, command line and original file name.
Wazuh Agent	Installed on the Windows client; forwards event logs to the Wazuh Manager.
Wazuh Manager and Dashboard	SIEM/XDR platform: receives logs, applies detection rules, generates alerts, runs active responses. The dashboard is used to search events and edit rules.
Shuffle	SOAR platform used to build the automation workflow (webhook, regex, API calls, email, user input).
TheHive 5	Case-management platform where the analyst reviews alerts. Uses Cassandra as its database and Elasticsearch for indexing.
VirusTotal	Open-source intelligence (OSINT) service used to enrich indicators of compromise (IOCs).
Mimikatz	Credential-dumping tool used in the lab to generate a realistic malicious event.
Linux cloud servers	Hosts for the Wazuh Manager and TheHive, administered over SSH.
Email	Channel used to notify the analyst and collect the approve/deny decision.

3. Architecture and Workflow

Figure 1 shows the overall architecture and Figure 2 shows the logical flow of data between the tools. The numbered steps below match the numbers in Figure 1.

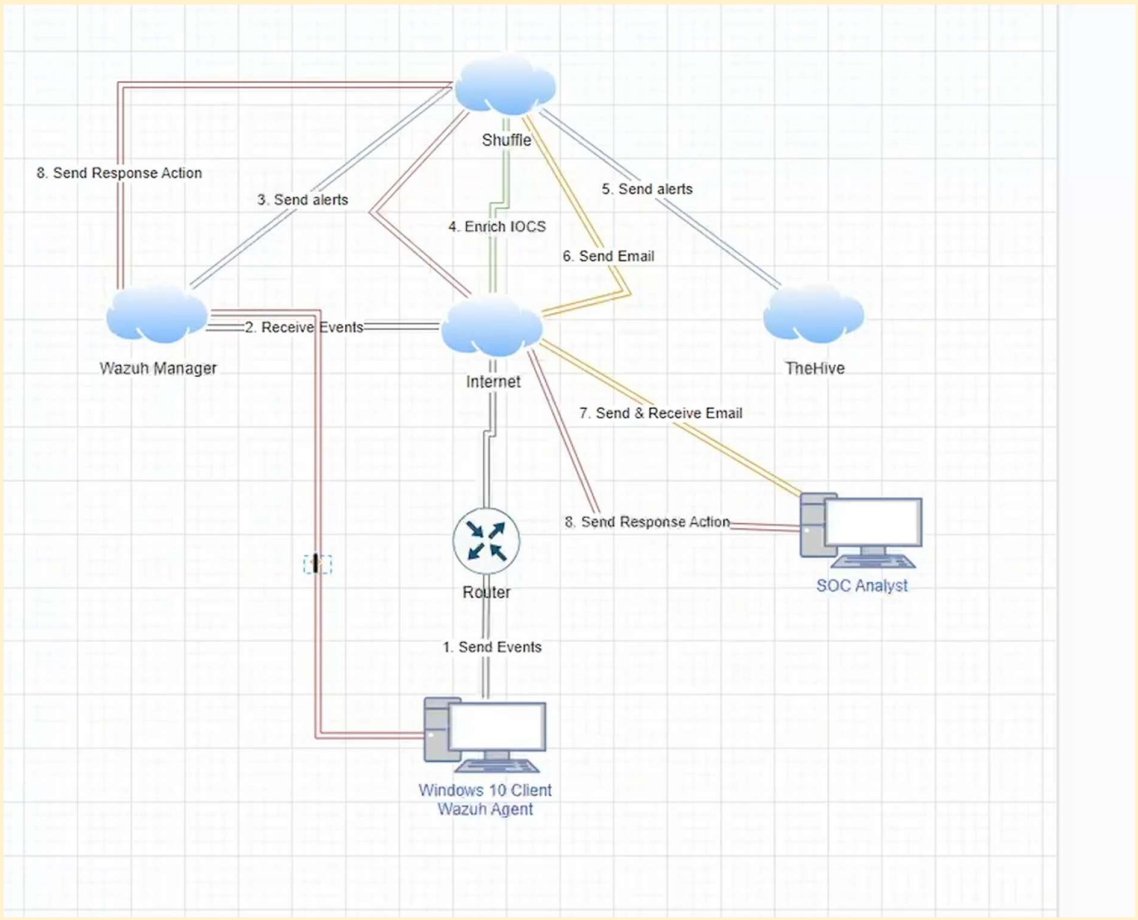


Figure 1: Network architecture and data flow of the SOC automation lab

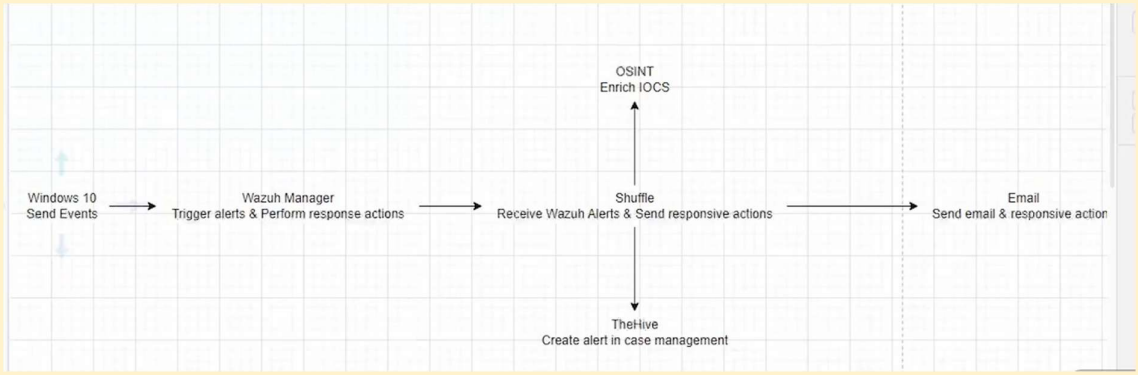


Figure 2: Logical workflow: Windows 10 > Wazuh Manager > Shuffle > TheHive / VirusTotal / Email

1. **Send events:** the Windows 10 client (Wazuh agent and Sysmon) sends its telemetry towards the Wazuh Manager.
2. **Receive events:** the Wazuh Manager receives the events and analyses them with its decoders and rules.

3. **Send alerts:** when a rule fires (the custom Mimikatz rule), the Wazuh integration forwards the alert to Shuffle through a webhook.
4. **Enrich IOCs:** Shuffle extracts the SHA256 hash from the alert and queries VirusTotal.
5. **Send alerts:** Shuffle creates an alert in TheHive so the case can be tracked and investigated.
6. **Send email:** Shuffle emails the SOC analyst with the alert details and a decision prompt.
7. **Send and receive email:** the analyst reads the email and answers using the TRUE / FALSE links.
8. **Send response action:** if approved, Shuffle sends an active-response command to the Wazuh Manager, which runs it on the endpoint.

4. Implementation

4.1 Setting up TheHive (Cassandra, Elasticsearch and TheHive)

TheHive depends on two back-end services, so these were configured first on the TheHive server over SSH.

Cassandra (database). The file /etc/cassandra/cassandra.yaml was edited. The listen_address was changed from the default to the server's own address so that the service is reachable by TheHive. Cassandra was then stopped, its default data in /var/lib/cassandra was removed so that it would start cleanly with the new settings, and the service was started again.

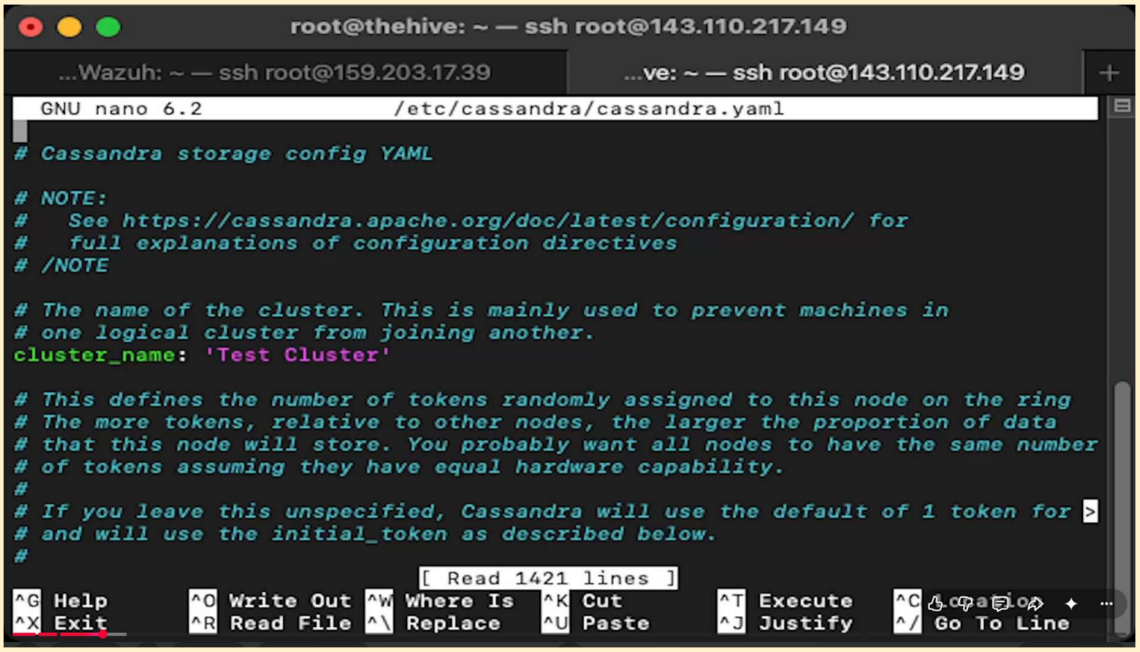


Figure 3: Editing the Cassandra configuration file (cassandra.yaml)



Figure 4: Cassandra listen_address set to the TheHive server's address

[INSERT FIGURE 5 HERE]
Screenshot PDF page 3 - bottom image (stop / rm / start commands)

Figure 5: Stopping Cassandra, clearing default data and restarting the service

Elasticsearch (indexing). In /etc/elasticsearch/elasticsearch.yml the HTTP port was kept at 9200 and cluster.initial_master_nodes was set to node-1 so that the single-node cluster could bootstrap itself.

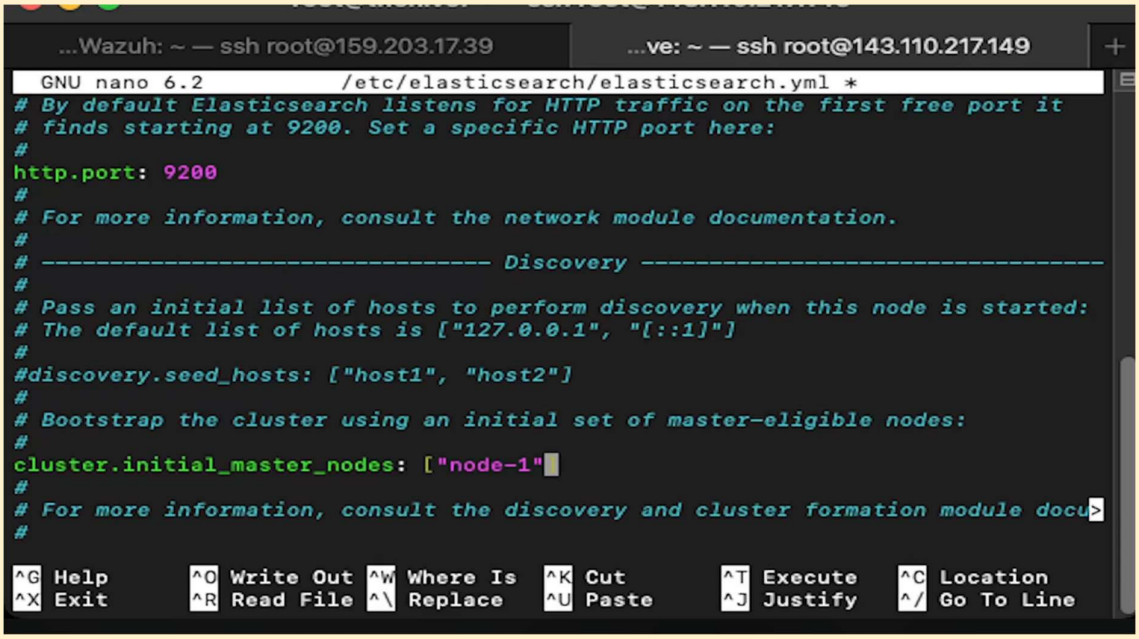


Figure 6: Elasticsearch configuration (elasticsearch.yml)

TheHive. In /etc/thehive/application.conf the database section (db.janusgraph) was pointed to Cassandra (cql backend, hostname set to the server address) and the index section was pointed to Elasticsearch. After saving, the thehive service was started, enabled to start on boot and its status was checked.

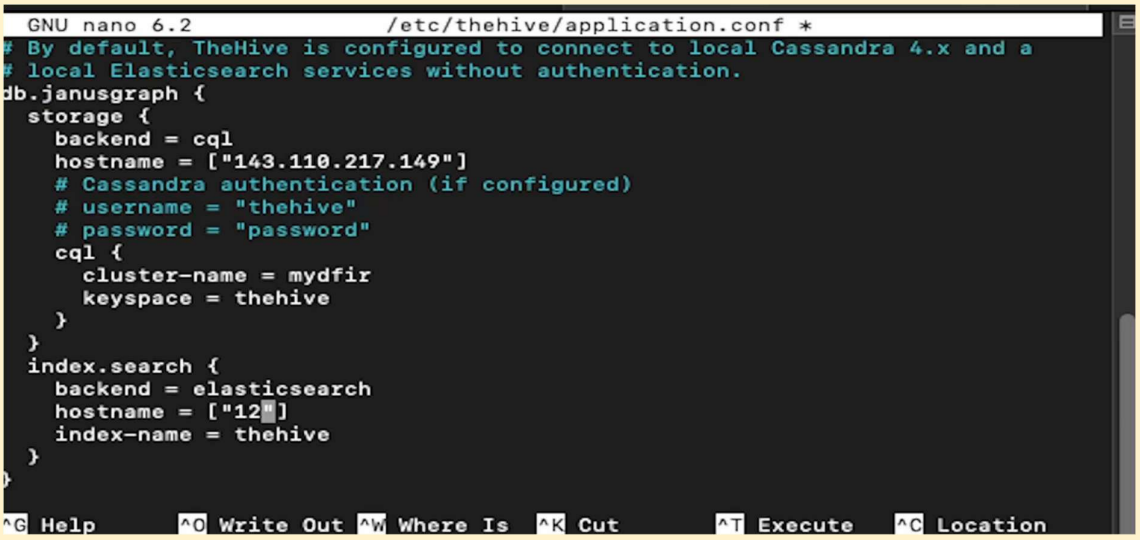


Figure 7: TheHive application.conf pointing to Cassandra and Elasticsearch

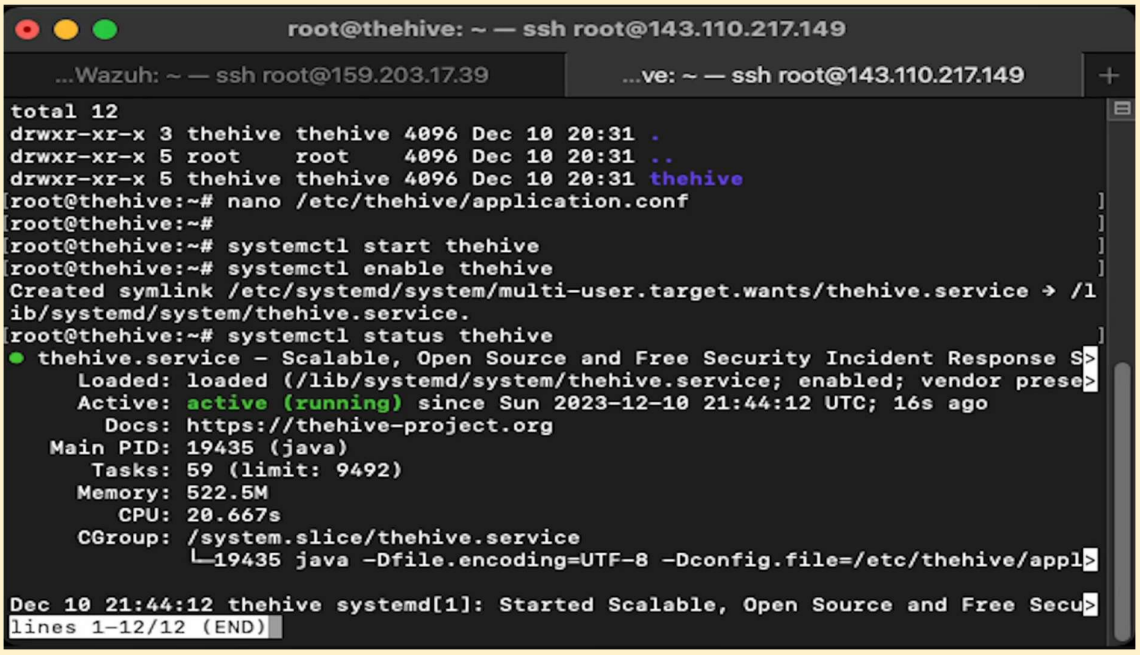


Figure 8: TheHive service started, enabled and running

4.2 Deploying the Wazuh agent on Windows 10

The Wazuh dashboard initially reported 0 agents. Using the Deploy new agent wizard, the Windows MSI package was selected, the Wazuh server address was entered, and the generated installation command was run on the Windows 10 client. Afterwards the dashboard showed one active agent, confirming that the endpoint was connected to the manager.

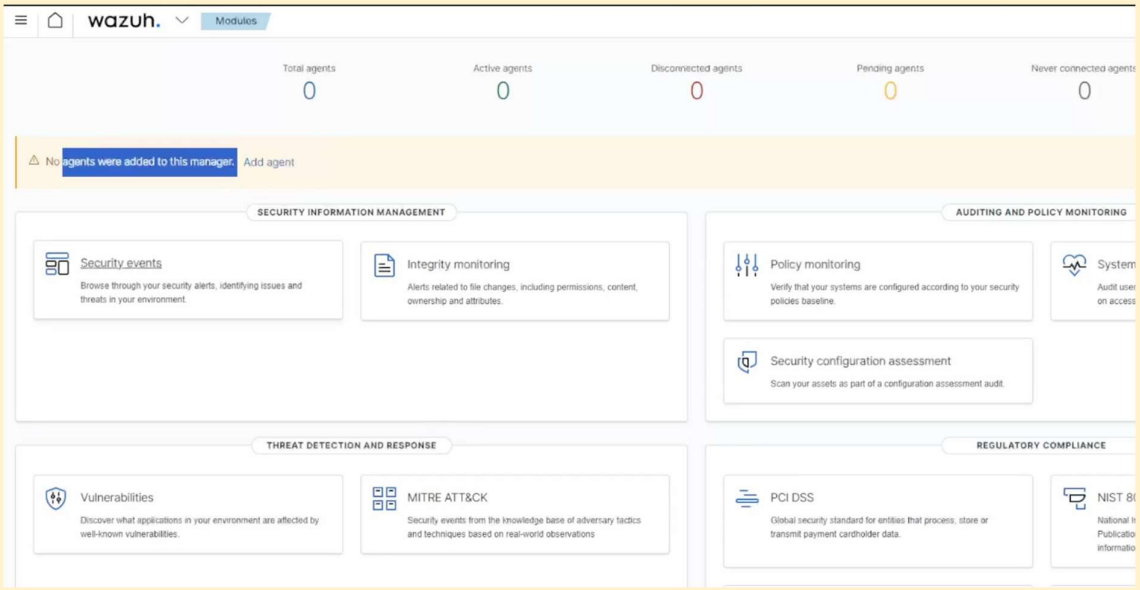


Figure 9: Wazuh dashboard before agent deployment (0 agents)

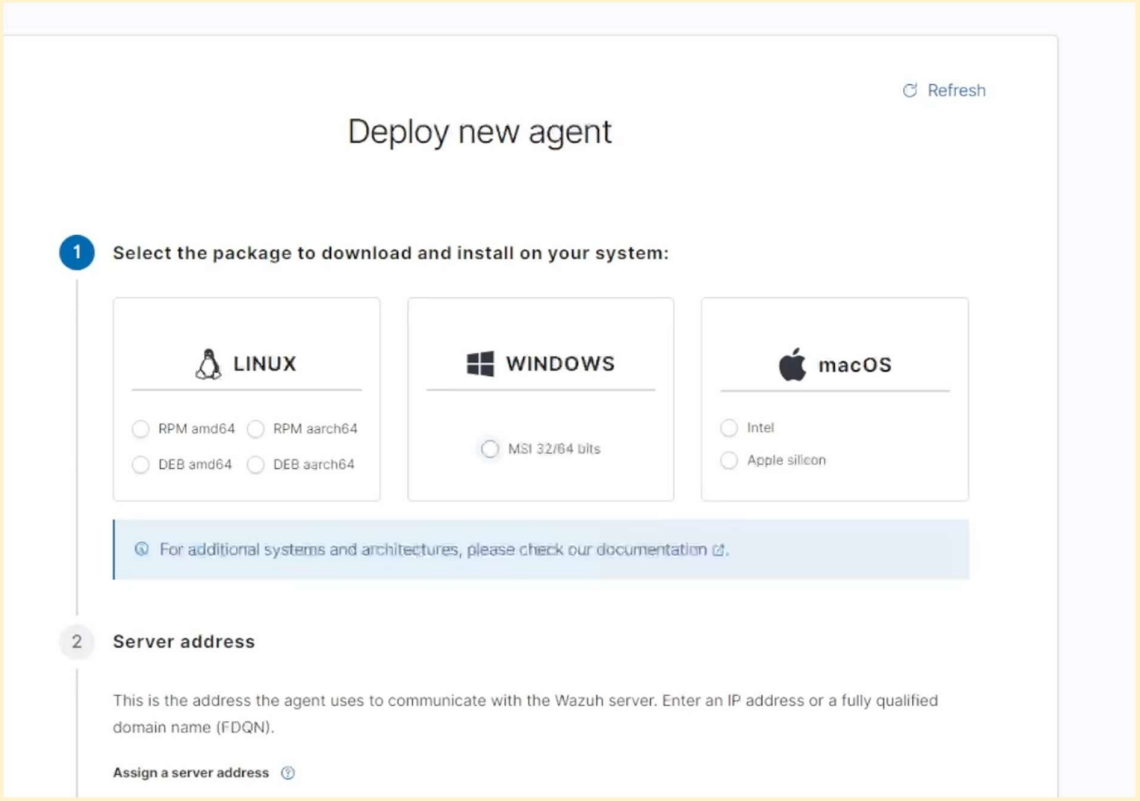


Figure 10: Wazuh Deploy new agent wizard

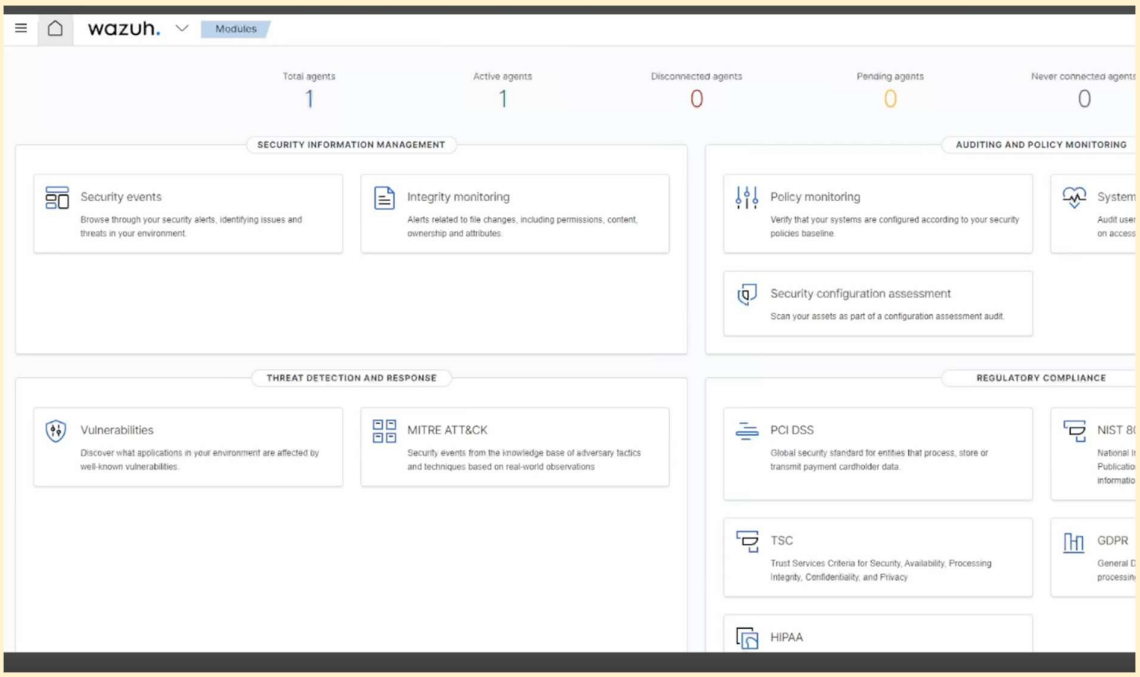


Figure 11: Wazuh dashboard showing the Windows 10 agent as active

4.3 Collecting Sysmon telemetry

By default the Wazuh agent collects the Application, Security and System event channels. Sysmon writes to its own channel (Microsoft-Windows-Sysmon/Operational), so the agent's ossec.conf was edited: an existing localfile block was duplicated and its location was changed to the Sysmon channel, with the eventchannel log format. After restarting the agent, Sysmon events started appearing in the Wazuh Security Events view.

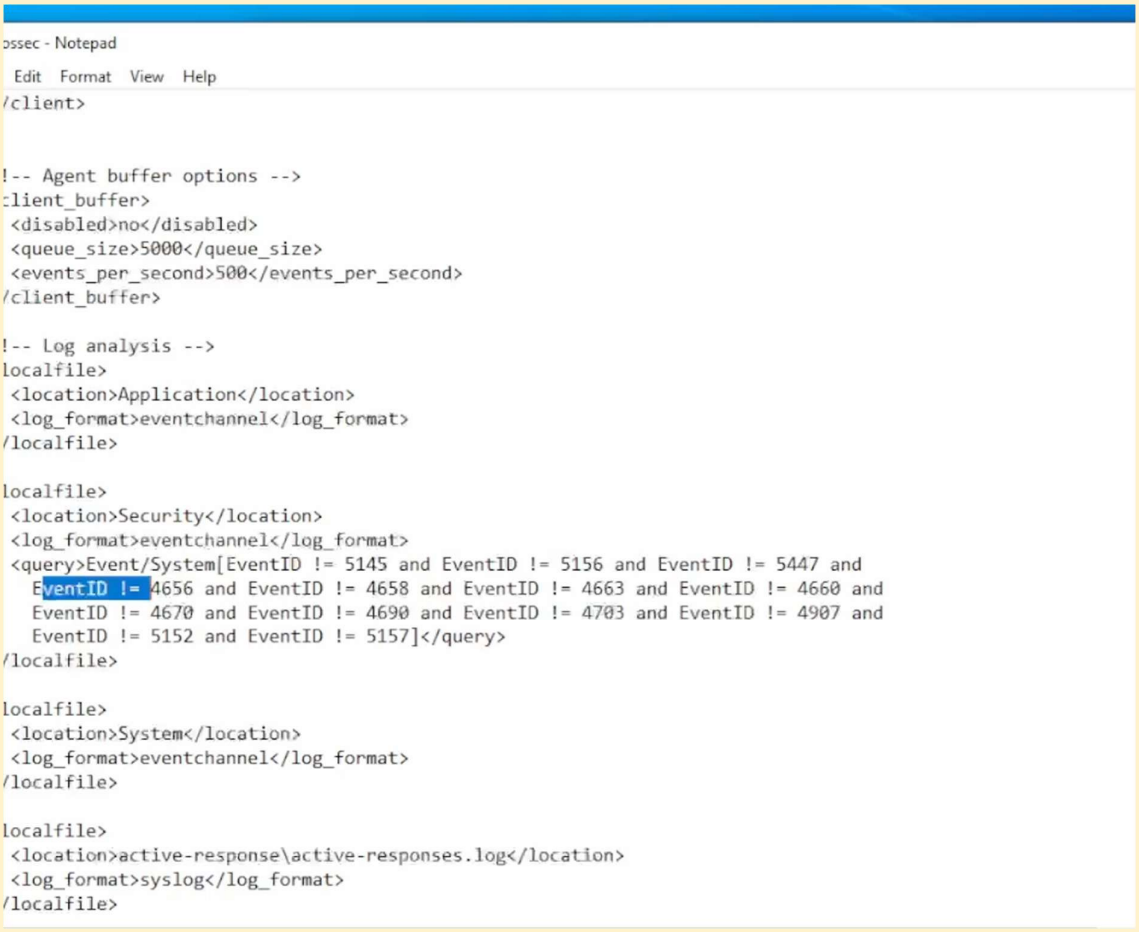


Figure 12: Windows agent ossec.conf - default log analysis section

*ossec - Notepad

File Edit Format View Help

```
  </client>

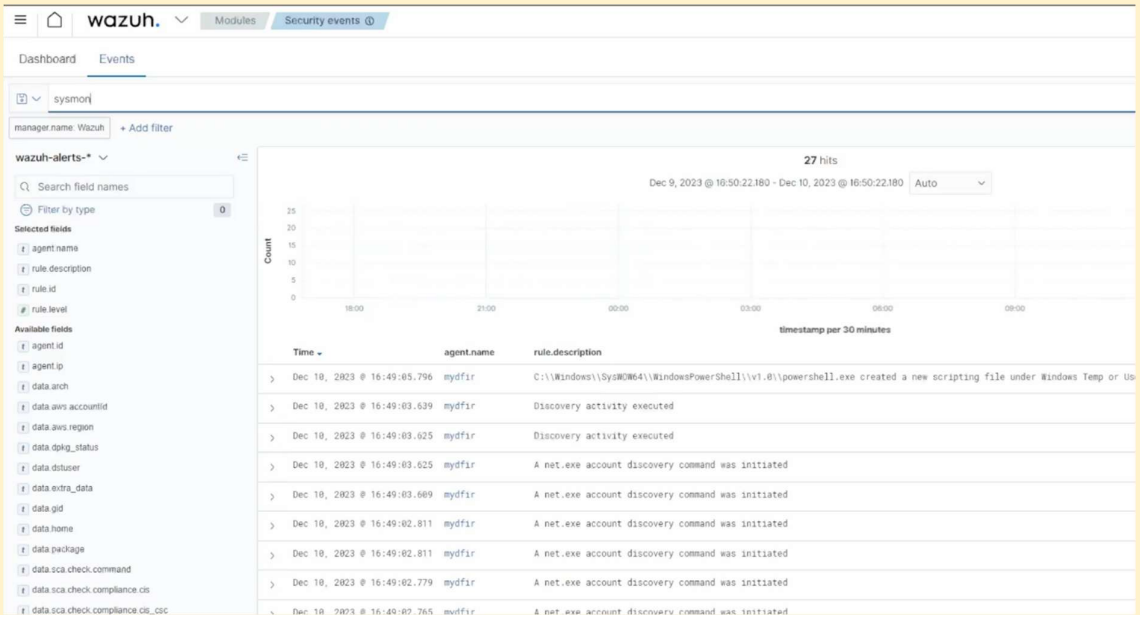

  <!-- Agent buffer options -->
  <client_buffer>
    <disabled>no</disabled>
    <queue_size>5000</queue_size>
    <events_per_second>500</events_per_second>
  </client_buffer>

  <!-- Log analysis -->
  <localfile>
    <location>Application</location>
    <log_format>eventchannel</log_format>
  </localfile>

  <localfile>
    <location>Security</location>
    <log_format>eventchannel</log_format>
    <query>Event/System[EventID != 5145 and EventID != 5156 and EventID != 5447 and
      EventID != 4656 and EventID != 4658 and EventID != 4663 and EventID != 4660 and
      EventID != 4670 and EventID != 4690 and EventID != 4703 and EventID != 4907 and
      EventID != 5152 and EventID != 5157]</query>
  </localfile>

<localfile>
    <location>Security</location>
    <log_format>eventchannel</log_format>
    <query>Event/System[EventID != 5145 and EventID != 5156 and EventID != 5447 and
      EventID != 4656 and EventID != 4658 and EventID != 4663 and EventID != 4660 and
      EventID != 4670 and EventID != 4690 and EventID != 4703 and EventID != 4907 and
      EventID != 5152 and EventID != 5157]</query>
  </localfile>
```

Figure 13: Adding a localfile block for the Sysmon channel in ossec.conf



)

Figure 14: Sysmon events visible in Wazuh Security Events

4.4 Simulating the attack and enabling event archiving

To generate a realistic detection scenario, Mimikatz was executed from PowerShell on the Windows 10 lab machine (an isolated test environment). Searching for "mimikatz" in the Wazuh dashboard returned no results. This is expected behaviour: Wazuh only stores events that match a rule as alerts, so the raw Sysmon event had not been indexed.

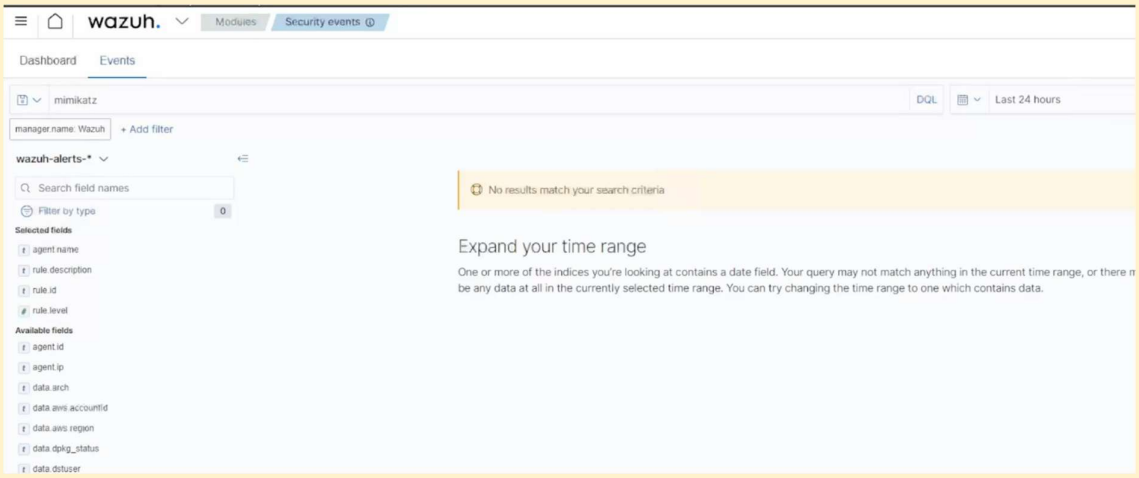


Figure 16: Dashboard search for "mimikatz" returns no results

To inspect the raw events, archiving was enabled on the Wazuh Manager. A backup of /var/ossec/etc/ossec.conf was made first, then the file was edited and the wazuh-manager service was restarted. The archives directory (/var/ossec/logs/archives) then contained archives.json and archives.log. A new index pattern for wazuh-archives-* was created in the dashboard, after which all events (351 in the last 24 hours) could be searched and the Mimikatz process-creation event could be examined for fields suitable for a detection rule.

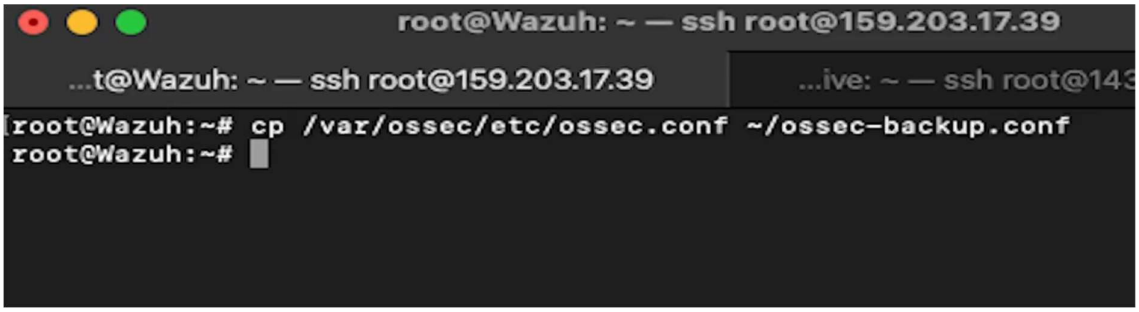


Figure 17: Backing up the Wazuh Manager's ossec.conf before editing

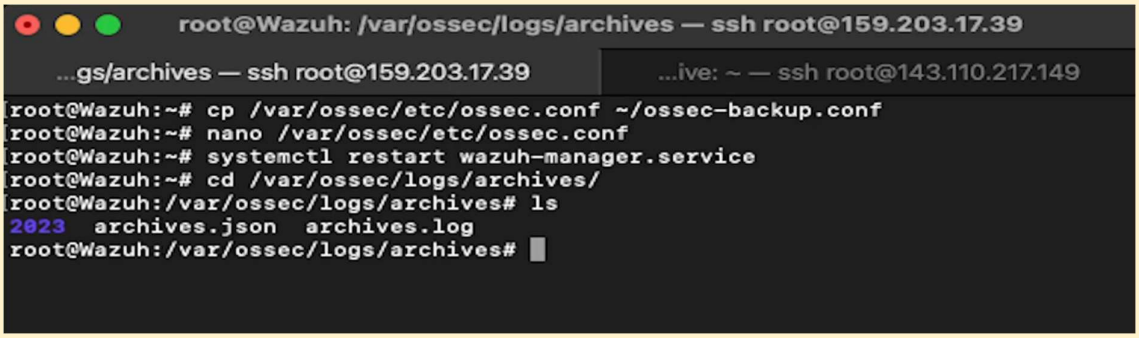


Figure 18: Restarting the Wazuh Manager and checking the archives directory

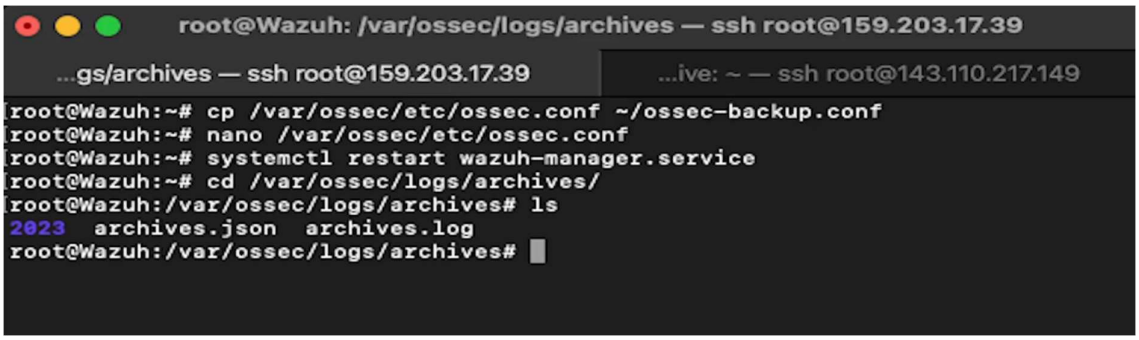


*Figure 20: Archived events searchable in wazuh-archives-**

4.5 Writing a custom detection rule

A custom rule was added to local_rules.xml through the Wazuh dashboard's rule editor. The rule (ID 100002, level 15) applies to Sysmon Event ID 1 (process creation) and matches the originalFileName field of the process against the regular expression (?i)mimikatz\.exe. It is described as "Mimikatz Usage Detected" and mapped to MITRE ATT&CK technique T1003 (OS Credential Dumping).

Why originalFileName? This value comes from the metadata embedded in the executable, not from the name on disk. An attacker who simply renames mimikatz.exe will not evade the rule. This was confirmed during testing, when the renamed executable was still detected. After saving, the manager was restarted because rule changes only take effect after a restart.

```
<!-- Local rules -->

<!-- Modify it at your will. -->
<!-- Copyright (C) 2015, Wazuh Inc. -->

<!-- Example -->
<group name="local,syslog,sshd,">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>

  <rule id="100002" level="15">
    <if_group>sysmon_event1</if_group>
    <field name="win.eventdata.originalFileName" type="pcre2">(?i)mim\.exe</field>
    <options>no_full_log</options>
    <description>Scripting interpreter spawned a new process</description>
    <mitre>
      <id>T1059.005</id>
    </mitre>
```

Figure 21: local_rules.xml - starting template for the custom rule

local_rules.xml

Changes will not take effect until a restart is performed.

```
<!-- Local rules -->

<!-- Modify it at your will. -->
<!-- Copyright (C) 2015, Wazuh Inc. -->

<!-- Example -->
<group name="local,syslog,sshd,">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>

  <rule id="100002" level="15">
    <if_group>sysmon_event1</if_group>
    <field name="win.eventdata.originalFileName" type="pcre2">(?i)mimikatz\.exe</field>
    <description>Mimikatz Usage Detected</description>
    <mitre>
      <id>T1003</id>
    </mitre>
  </rule>

</group>
```

Figure 22: Final custom rule: Mimikatz Usage Detected (MITRE T1003)

4.6 Building the Shuffle automation workflow

Workflow and webhook. A new workflow named "SOC Automation Project" was created in Shuffle. A Webhook trigger (Wazuh-Alerts) was added and started, which provides the URL that Wazuh sends alerts to.

Wazuh integration. On the Wazuh Manager an integration block was added to ossec.conf with the name shuffle, the webhook URL as hook_url, alert_format set to json, and rule_id set to 100002 so that only the Mimikatz detection is forwarded to Shuffle instead of every alert.

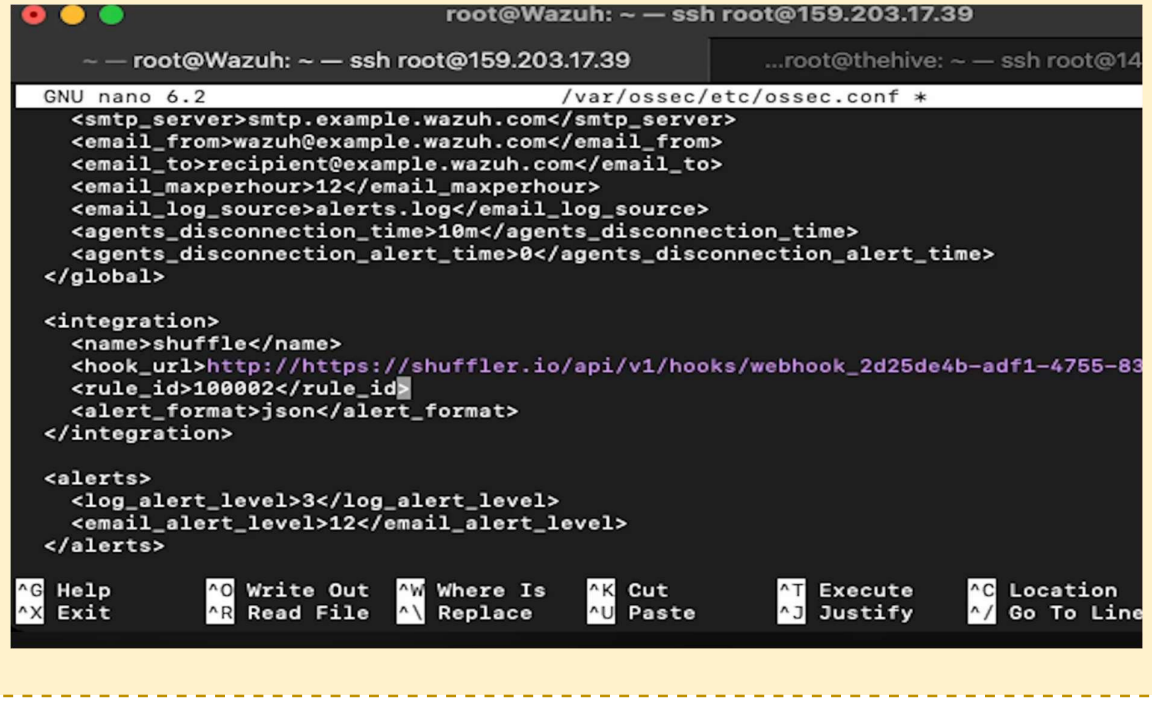


Figure 25: Wazuh integration block sending rule 100002 alerts to Shuffle

Verifying alert delivery. Re-running Mimikatz produced an execution in Shuffle. The execution argument shows a successful run containing the alert title "Mimikatz Usage Detected" and the full Sysmon event data (process ID, image, command line, hashes and computer name), confirming that Wazuh and Shuffle were connected correctly.

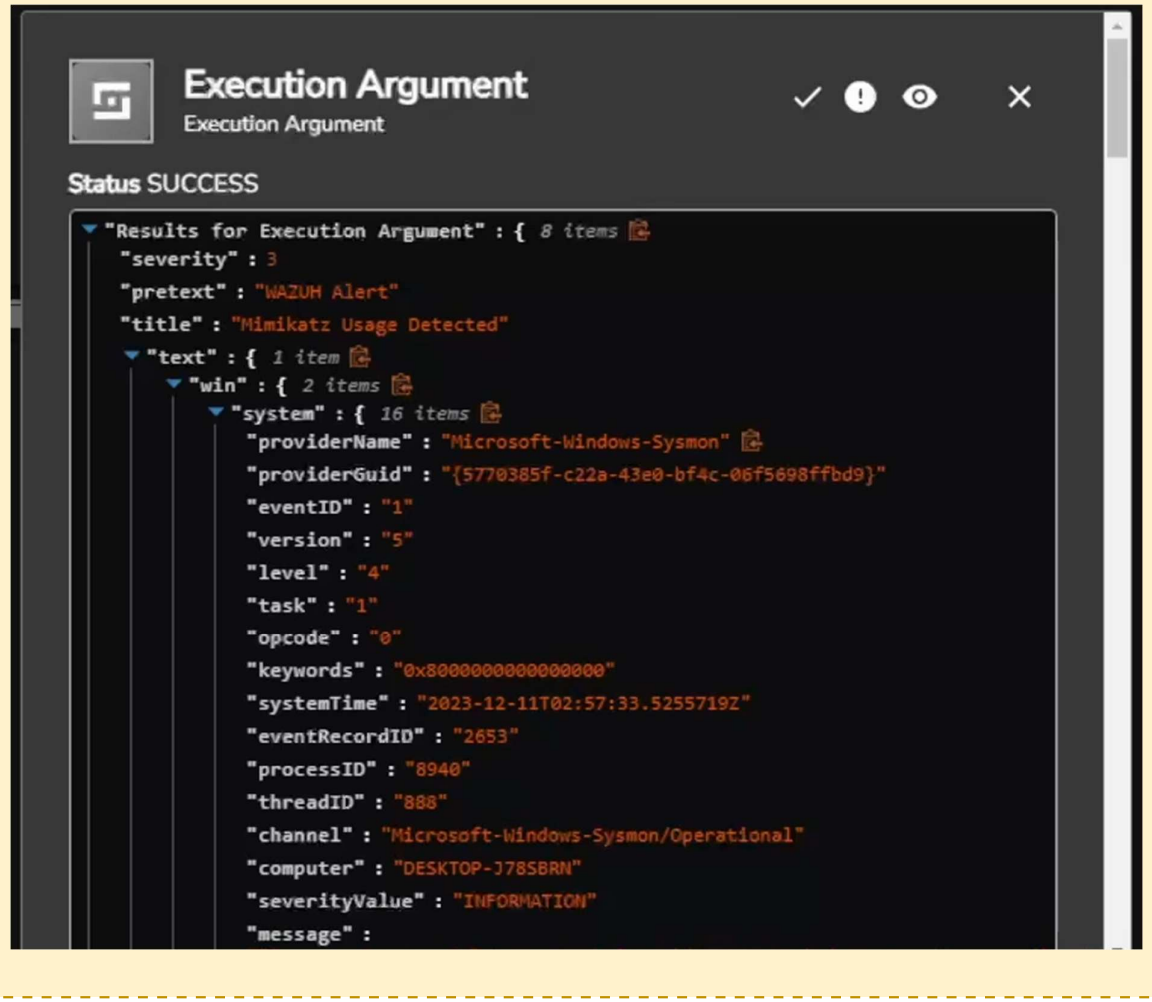


Figure 27: Mimikatz alert received by Shuffle (execution argument)

Extracting the IOC. A regex node (SHA256-Regex) pulls the SHA256 file hash out of the alert so that it can be used as an indicator.

Threat-intelligence enrichment. The VirusTotal v3 app was added to query the extracted hash. The API response is visible in the execution view. In this run the lookup returned a 404 NotFoundError ("Resource not found"), which means the hash was not present in VirusTotal's database. The node still completed and passed its result on to the next step.

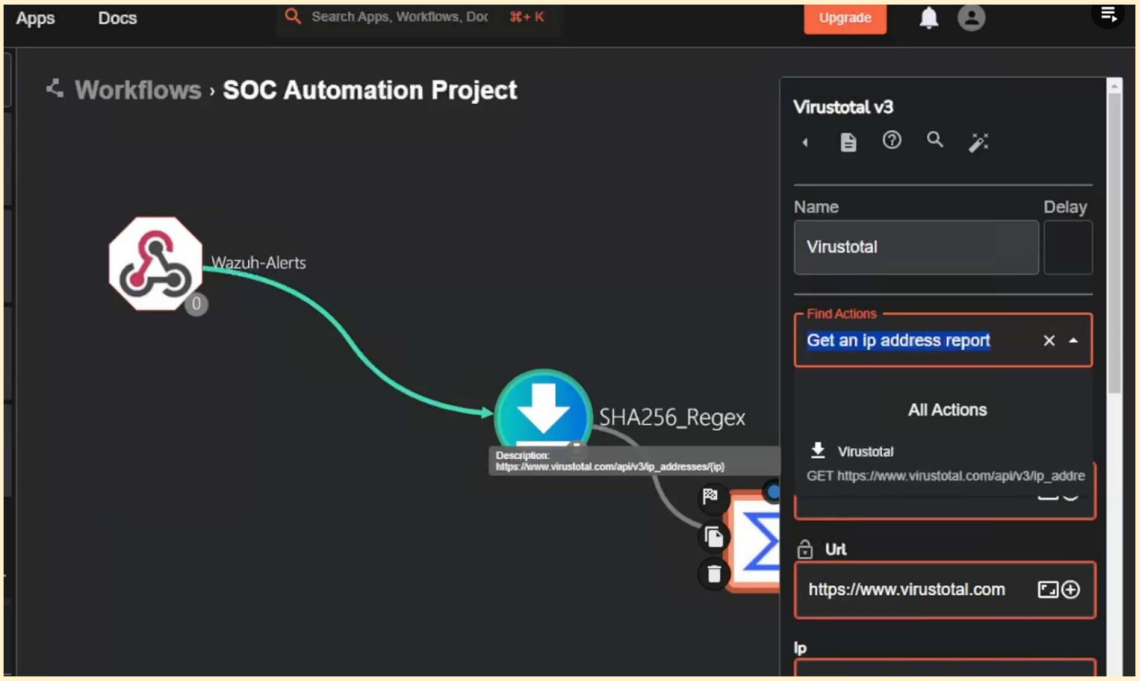


Figure 28: Configuring the VirusTotal app in Shuffle

Case creation in TheHive. The TheHive 5 app in Shuffle was authenticated with an API key and the URL of the TheHive instance. The workflow then creates an alert in TheHive containing a title, the MITRE technique tag (T1003), a description naming the affected host and user, and a summary with the process ID and command line. This gives the analyst a ready-made case instead of a raw log line.

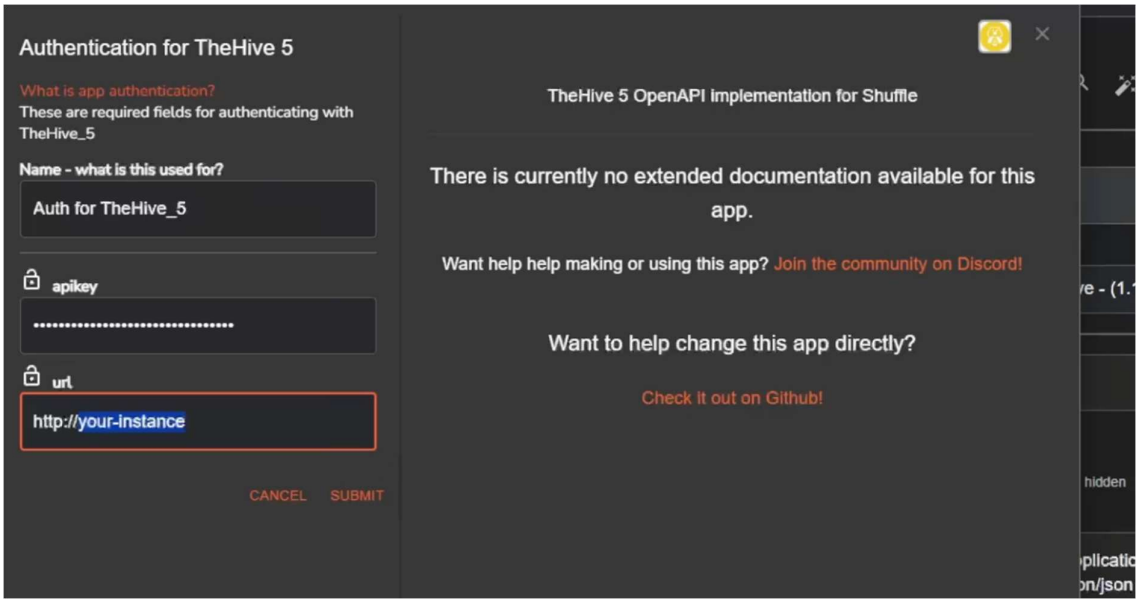


Figure 30: Authenticating the TheHive 5 app in Shuffle



Figure 31: Alert created in TheHive with MITRE tag, description and summary

Analyst notification. An email node sends the analyst a message that includes the details and asks whether the source IP should be blocked, with a TRUE link and a FALSE link. Clicking a link feeds the answer back into the workflow.

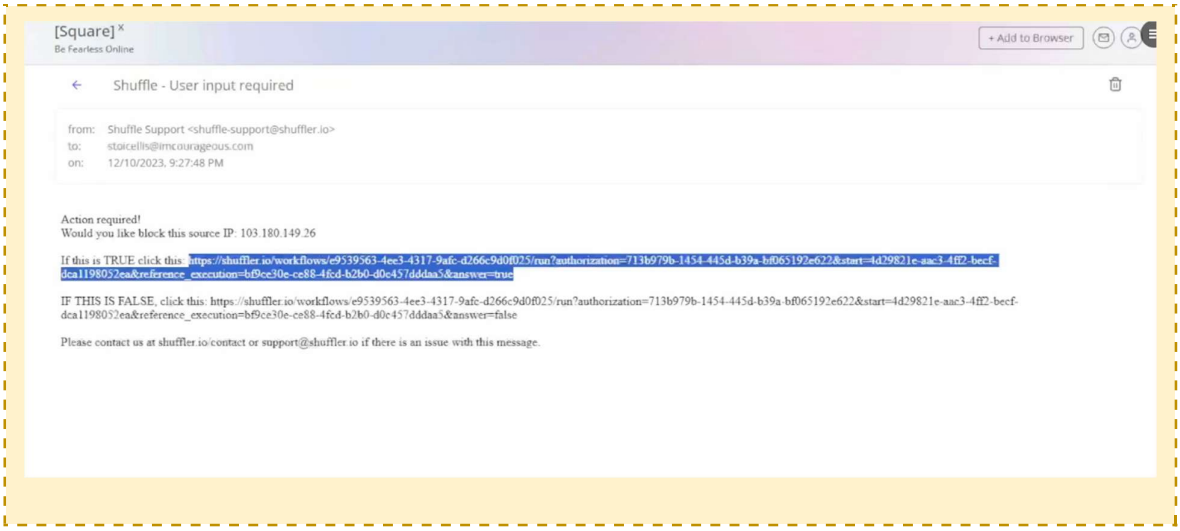


Figure 32: Analyst email from Shuffle asking whether to block the IP

Completed workflow. Figure 33 shows all nodes of the finished workflow: the Wazuh-Alerts webhook, a Get-API node, SHA256 regex extraction, VirusTotal enrichment, TheHive alert creation, the email node and the Wazuh response node.

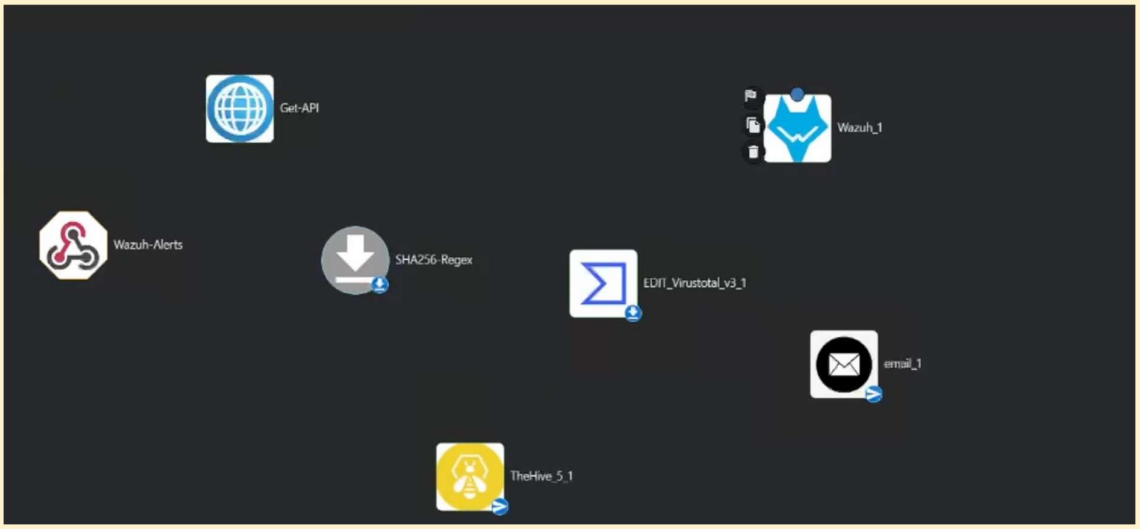


Figure 33: Completed Shuffle workflow

4.7 Active response (blocking from Shuffle)

On the Wazuh Manager, the active-response section of ossec.conf was configured with the firewall-drop command (location: local, level 5). From Shuffle, the Wazuh app's run-command action was used to send firewall-drop to the endpoint (agent 002) with a test IP address as its argument. The Wazuh API confirmed success: one affected item, zero failed items, and the message that the active-response command was sent.

```
<command>
  <name>netsh</name>
  <executable>netsh.exe</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <level>5</level>
  <timeout>no</timeout>
</active-response>

<!-- Log analysis -->
<localfile>
  <log_format>command</log_format>
  <command>df -P</command>
  <frequency>360</frequency>
```

Figure 34: Active-response configuration on the Wazuh Manager

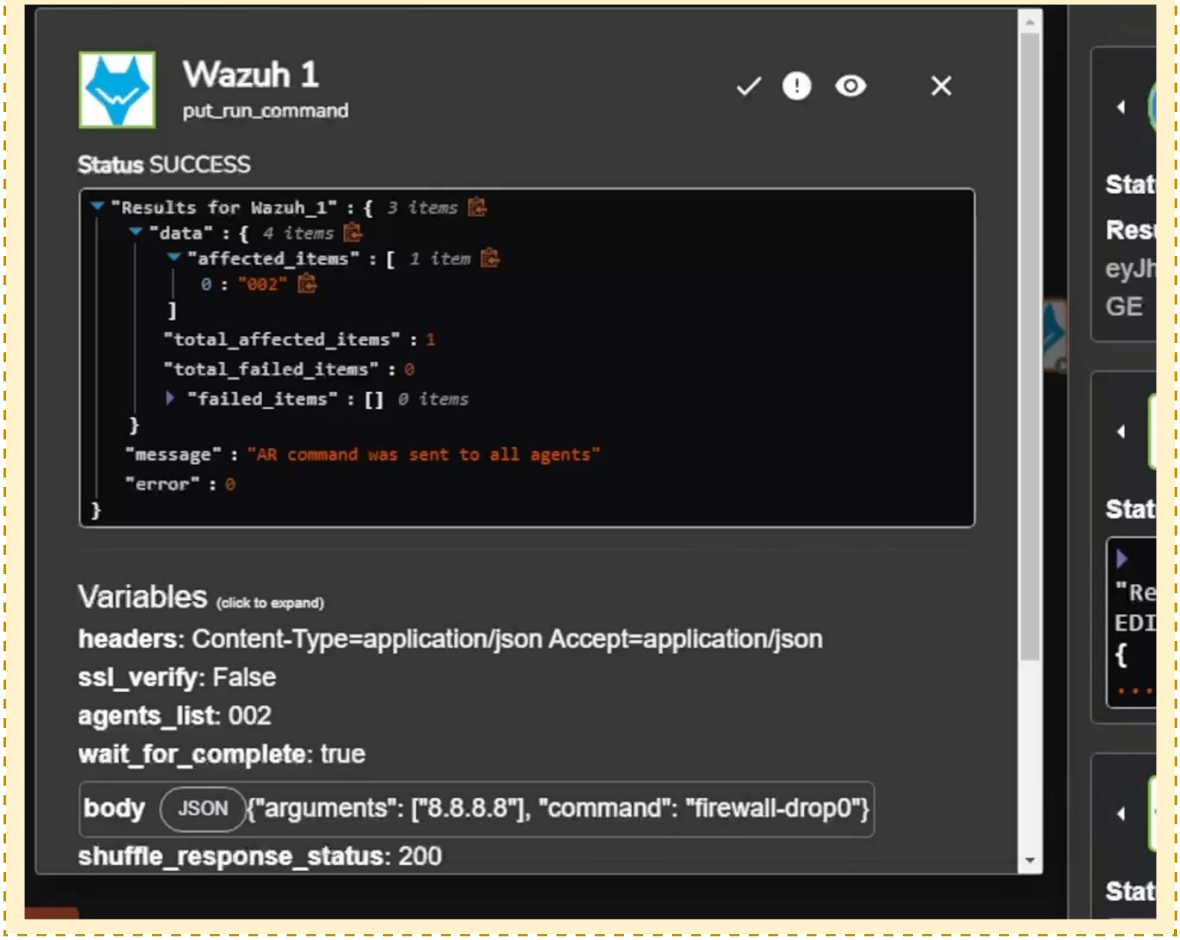


Figure 35: Shuffle triggering the Wazuh active-response command

5. Results

The pipeline worked end to end:

- Executing Mimikatz on the Windows 10 endpoint generated a Sysmon process-creation event that was collected by the Wazuh agent.
- The custom rule 100002 fired at level 15 and was tagged with MITRE ATT&CK T1003.
- The alert was delivered automatically to Shuffle, which extracted the file hash and queried VirusTotal.
- A fully populated alert appeared in TheHive without any manual data entry.
- The analyst received an email with a one-click approve/deny decision.
- A response command was successfully sent through Wazuh to block an IP address on the endpoint.

6. Challenges and Troubleshooting

Issue	Cause	Resolution
Searching for "mimikatz" returned no results	Wazuh only indexes events that match a rule as alerts, so the raw Sysmon event was not stored.	Enabled event archiving on the manager, created a wazuh-archives-* index pattern and inspected the raw event to build the rule.
New rule had no effect at first	Wazuh loads rule changes only when the manager restarts.	Restarted the wazuh-manager service after editing local_rules.xml.
VirusTotal lookup returned 404	The queried hash was not found in VirusTotal's database (or the request needed adjusting).	Reviewed the raw response in Shuffle's execution view and adjusted the VirusTotal node (the workflow uses an edited copy of the app).

7. Skills and Learning Outcomes

- **SIEM operations:** deploying agents, searching events, working with indexes and archives in Wazuh.
- **Detection engineering:** writing a custom rule with a regular expression, severity level and MITRE ATT&CK mapping.
- **SOAR automation:** building a workflow with webhooks, regex parsing, API calls, email and user-input steps in Shuffle.
- **Threat intelligence:** enriching indicators of compromise with VirusTotal.
- **Incident management:** creating structured alerts and cases in TheHive.
- **Incident response:** configuring and triggering Wazuh active response with analyst approval.
- **Linux administration:** editing configuration files, managing services with systemctl, working over SSH.

8. Security Considerations and Future Improvements

8.1 Security considerations

- Mimikatz was run only inside an isolated lab machine created for testing.
- API keys were entered through Shuffle's authentication fields rather than written into workflow logic.
- In a production setting, webhook URLs should use HTTPS, server ports should be restricted with firewall rules, and automated blocking should be limited to approved IP addresses.

8.2 Future improvements

- Add detection rules for other techniques, such as suspicious PowerShell use or brute-force logins.
- Enrich IP addresses as well as file hashes, and use the reputation score to decide automatically whether to ask the analyst.
- Add chat notifications (Slack or Microsoft Teams) in addition to email.
- Use time-limited firewall blocks and automatically close TheHive cases after resolution.

- Extend the lab with a network sensor such as Suricata for network-level detection.

9. Conclusion

This project demonstrates how a SIEM, a SOAR platform and a case-management system can be combined into an automated SOC workflow. A single detection, Mimikatz execution, triggered enrichment, case creation, analyst notification and a controlled response without manual log hunting. The project gave practical experience with the tools and processes used in real security operations and forms a foundation for building more detections and automations in the future.

10. References

- MyDFIR - SOC Automation Project walkthrough (YouTube): the design and workflow this project is based on.
- Wazuh documentation - https://documentation.wazuh.com
- Shuffle documentation - https://shuffler.io/docs
- TheHive Project - https://thehive-project.org
- MITRE ATT&CK, T1003 OS Credential Dumping - https://attack.mitre.org/techniques/T1003/
- Microsoft Sysinternals, Sysmon - https://learn.microsoft.com/sysinternals/downloads/sysmon
- VirusTotal API v3 - https://docs.virustotal.com

Appendix: Screenshot Placement Guide (delete before submitting)

Each yellow box in the document marks where a screenshot goes. "Page" refers to the page of your SOC_AUTOMATION_PROJECT.pdf, and the position tells you which image on that page to use. Replace each yellow box with the screenshot and keep the caption underneath.

Figure	PDF page	Which image on that page	What it shows
Figure 1	1	top image (network architecture diagram)	Network architecture and data flow of the SOC automation lab
Figure 2	1	bottom image (logical workflow strip)	Logical workflow: Windows 10 > Wazuh Manager > Shuffle > TheHive / VirusTotal / Email
Figure 3	2	middle image (cassandra.yaml open in nano)	Editing the Cassandra configuration file (cassandra.yaml)
Figure 4	3	top image (listen_address)	Cassandra listen_address set to the TheHive server's address
Figure 5	3	bottom image (stop / rm / start commands)	Stopping Cassandra, clearing default data and restarting the service
Figure 6	4	top image (elasticsearch.yml)	Elasticsearch configuration (elasticsearch.yml)
Figure 7	4	bottom image (application.conf)	TheHive application.conf pointing to Cassandra and Elasticsearch
Figure 8	5	top image (systemctl status thehive)	TheHive service started, enabled and running
Figure 9	5	bottom image (dashboard with 0 agents)	Wazuh dashboard before agent deployment (0 agents)
Figure 10	6	top image (Deploy new agent)	Wazuh Deploy new agent wizard
Figure 11	6	bottom image (1 active agent)	Wazuh dashboard showing the Windows 10 agent as active
Figure 12	7	bottom image (Notepad, ossec.conf log analysis section)	Windows agent ossec.conf - default log analysis section
Figure 13	8	top image (duplicated localfile block)	Adding a localfile block for the Sysmon channel in ossec.conf
Figure 14	8	bottom image (search: sysmon, 27 hits)	Sysmon events visible in Wazuh Security Events
Figure 15	9	top image (PowerShell running mimikatz.exe)	Mimikatz executed on the Windows 10 lab endpoint
Figure 16	9	middle image (no results found)	Dashboard search for "mimikatz" returns no results
Figure 17	9	bottom image (cp ossec.conf backup)	Backing up the Wazuh Manager's ossec.conf before editing
Figure 18	10	top image (restart and archives directory)	Restarting the Wazuh Manager and checking the archives directory
Figure 19	10	bottom image (index patterns)	Default Wazuh index patterns (the archives pattern was added afterwards)
Figure 20	11	top image (351 hits)	Archived events searchable in wazuh-archives-*

Figure	PDF page	Which image on that page	What it shows
Figure 21	11	bottom image (initial local_rules.xml)	local_rules.xml - starting template for the custom rule
Figure 22	12	top image (final rule)	Final custom rule: Mimikatz Usage Detected (MITRE T1003)
Figure 23	12	bottom image (Security Alerts table)	Security alerts in Wazuh after loading the new rule
Figure 24	13	only image (New workflow dialog)	Creating the "SOC Automation Project" workflow in Shuffle
Figure 25	15	top image (integration block with hook_url and rule_id)	Wazuh integration block sending rule 100002 alerts to Shuffle
Figure 26	15	bottom image (webhook trigger in Shuffle)	Shuffle webhook trigger (Wazuh-Alerts) connected to the workflow
Figure 27	16	only image (Execution Argument, status SUCCESS)	Mimikatz alert received by Shuffle (execution argument)
Figure 28	17	only image (VirusTotal app configuration)	Configuring the VirusTotal app in Shuffle
Figure 29	18	only image (hash report, 404)	VirusTotal response for the extracted hash
Figure 30	19	top image (Authentication for TheHive 5)	Authenticating the TheHive 5 app in Shuffle
Figure 31	19	bottom image (alert in TheHive)	Alert created in TheHive with MITRE tag, description and summary
Figure 32	22	bottom image (Shuffle user input email)	Analyst email from Shuffle asking whether to block the IP
Figure 33	20	bottom image (workflow including Wazuh_1)	Completed Shuffle workflow
Figure 34	21	only image (active-response configuration)	Active-response configuration on the Wazuh Manager
Figure 35	22	top image (Wazuh_1 put_run_command result)	Shuffle triggering the Wazuh active-response command

Screenshots you can skip (duplicates or not needed): page 2 top (SSH prompt) and page 2 bottom (duplicate of the middle image), page 3 middle (partial version of the bottom image), page 7 top (duplicate of the bottom image), both images on page 14 (blank workflow and the unfilled integration template), and page 20 top (workflow before the Wazuh node was added).

Before you submit: blur or crop the webhook ID in the hook_url (Figures 25 and 26), server IP addresses, and the email addresses in Figure 32 so no private details or secrets are exposed.